Student Assignment Brief

This document is intended for Coventry University Group students for their own use in completing their assessed work for this module. It must not be passed to third parties or posted on any website. If you require this document in an alternative format, please contact your Module Leader.

Contents:

The work you submit for this assignment must be your own independent work, or in the case of a group assignment your own groups' work. More information is available in the 'Assignment Task' section of this assignment brief.

Assignment Information

Module Name: Integrative Project

Module Code: 5026CMD

Assignment Title: Student Information Management System (SIMS)

Assignment Due: Week 12

VIVA Presentation : Week 13 – Week 15

Assignment Credit: 10 credits

Word Count (or equivalent): N/A

Assignment Type: Portfolio

Percentage Grade (Applied Core Assessment). You will be provided with an overall grade between 0% and 100%. To pass the assignment you must achieve a grade of 40% or above.

Assignment Task

This coursework assesses learning outcomes 1 and 2 and represents 10 credits of the module. This assignment is a Portfolio. You will work in a group of 5, but the portfolio is marked individually.

Detail of Assessment

Students will work in groups to develop a Web-Based Student Information Management System (SIMS). Each student will be assessed based on their individual contributions to system development, including their source code implementation.

Final submissions must clearly demonstrate personal involvement and will be verified through an individual viva session to confirm understanding, technical competency, and ownership of work.

Each student must submit an individual academic report supported by formal citations and references.

Although the system is developed collaboratively in groups, all submissions and evaluations are individual. Any shared components must be clearly identified and justified within the report.

Coursework Brief

A college currently manages student records, course registration, attendance tracking, and academic information using manual or partially digital processes. These practices result in data redundancy, delayed record updates, and difficulty in monitoring student academic performance and administrative activities.

The objective of this project is to develop a Student Information Management System (SIMS) that automates academic and administrative operations, including:

- Student registration and record management
- Course enrolment
- Academic performance tracking
- Attendance monitoring
- Fee management
- Academic reporting and analytics

The system aims to improve data accuracy, administrative efficiency, decision-making, and student experience.

Coursework Task:

You are required to develop a Student Information Management System (SIMS) using the C# programming language. The system must include a user-friendly Graphical User Interface (GUI) and be deployable as a Web-based application.

The system must support multiple user roles:

- Head of Programme (Admin)
- Lecturer
- Student

System Requirements :

1. Head of Programme (Admin) Module :
    a. Log in and manage academic programmes, and courses information.
    b. Register lecturers and students and assign user roles.
    c. Manage student admission and enrolment records.
    d. Monitor overall student statistics and academic performance summaries.
    e. Generate institutional reports such as enrolment statistics, student performance reports, and attendance summaries.
    f. Manage academic calendar and announcements.

2. Lecturer Module:
    a. Log in and manage personal profile information.
    b. View assigned courses and registered students.
    c. Record and update student attendance.
    d. Enter, update, and publish assessment marks or grades.
    e. Monitor student academic progress.
    f. Post announcements or course materials to students.
    g. Identify students with poor attendance or academic performance.

3. Student Module:
    a. Register and log in securely to the system.
    b. View personal academic profile and enrolled courses.
    c. Register or drop courses within permitted periods.
    d. View attendance records and academic results.
    e. Track academic history and performance reports.
    f. Receive real-time notifications regarding announcements, grades, and academic updates.

Coursework Performance Levels :

Level	Description
Basic	1. Student registration and login. 2. Manual course enrolment. 3. Basic student profile management. 4. Simple attendance recording. 5. Basic academic record display and transaction logs.
Intermediate	1. Automated course enrolment validation. 2. Attendance percentage calculation. 3. Professional result slip or transcript generation. 4. Real-time grade updates. 5. Filtered reports (student list, course enrolment, attendance reports). 6. Report generation using date or semester filtering.
Advanced	1. Role-based access control (HOP/Admin, Lecturer, Student). 2. GPA/CGPA automatic calculation. 3. Academic analytics dashboards (performance trends, risk students). 4. Export reports in CSV, PDF, or Excel formats. 5. Interactive admin dashboard with charts and visual analytics. 6. Email or system notifications for results, announcements, and attendance alerts. 8. Automated academic warning system for low performance students.

Must use the technology stack below:

Frontend: ASP.NET

Backend: C# Programming Language, Visual Studio IDE

Database: SQL Server Management Studio

Individual Portfolio Requirements :

Each student is required to submit an individual Portfolio documenting their contribution, analysis, design decisions, development process, and testing activities carried out throughout the project lifecycle.

The portfolio must demonstrate the application of Software Engineering principles, structured documentation practices, and individual technical understanding.

Your portfolio must include the following components: :

- Cover page
- Table of contents
- List of figures and tables
- **Chapter 1 : Introduction**
    - Project Overview
        - Brief description of the system developed.
        - Purpose and scope of the project.
        - Target users.
        - System scope and boundaries.
    - Background of the Problem
        - Description of current issues or limitations in existing/manual systems.
        - Motivation for developing the proposed solution.
    - Research Question
        - Key problem statements addressed by the system.
        - Questions guiding system development.
    - Project Objectives
        - Clear and measurable objectives aligned with software engineering practices.

- **Chapter 2 : System Architecture & Quality Evaluation**
    - Proposed System Architecture
    - Architectural Design Decisions
    - Architecture Evaluation Against Quality Attributes

- **Chapter 3 : Software Project Management**
    - System Development Methodology
        - Selected SDLC model (Agile/Waterfall/V-model, etc.).
        - Justification for choosing the model.
    - Project Planning
        - Work Breakdown Structure (WBS)
        - Gantt Chart showing project phases, milestones and task distribution.
    - Configuration Management
        - Version control concept
    - Tools used : GitHub
    - Project Collaboration Tools – GitHub, Trello, etc.

- **Chapter 4 : Tools & Technology Selection**
    - Technology Stack
        - Frontend
        - Backend
        - Database
    - Justification of selected tools
    - System Design Models
        - Use Case Diagram
        - Class diagram
        - Sequence diagram
        - Database ER diagram

- **Chapter 5 : System Development**
    - Module implementation
    - UI/UX Design Principles
    - Database Design

- **Chapter 6 : Software Quality Assurance**
    - Software Quality Standards Applied
    - Testing Strategy
    - Test Plan
    - Sample Test Cases
    - Quality Assurance Tools

- **Chapter 7 : Evaluation & Reflection**
    - System Evaluation
    - Challenges Faced
    - Future Improvements
    - Self-Reflection & Learning outcomes.

- References
- Appendices (Source code samples, GitHub commits, Interface Screenshots, test results and additional diagrams).

Note : Portfolio format and expectations will be discussed during lecture/practical classes. Final portfolio must reflect individual effort and professionalism to achieve higher marks. Clearly indicate any shared/group developed components with justification.

Submission Instructions:

All submissions will be checked against each other and the internet for possible plagiarism.

Student Evaluation :

Portfolio	50%

Submission:

1. Individual Portfolio:
    a. You should submit a single PDF file with a filename in the following format: FullName_INTIID_SubjectCode_AssignmentName
    Thirunageswari_P2012345_5026CMD_Report.pdf

2. You are also required to submit the GitHub link to where you did the work. Branches, commits, pull request and merge pull request will be checked, so a single upload of the working code will give you less marks. If the GitHub is not accessible or the files do not exist, it will affect your marks.

Marking and Feedback

How will my assignment be marked?

Your assignment will be marked by the module leader.

How will I receive my grades and feedback?

Provisional marks will be released once internally moderated.

Feedback will be provided by the module leader alongside grades release.

Marks and feedback can be accessed by going to the Grades on Canvas system.

Your provisional marks and feedback should be available by end of the semester.

What will I be marked against?

Details of the marking criteria for this task can be found at the bottom of this assignment brief.

Assessed Module Learning Outcomes

The Learning Outcomes for this module align to the marking criteria which can be found at the end of this brief. Ensure you understand the marking criteria to ensure successful achievement of the assessment task. The following module learning outcomes are assessed in this task:

1. Compare an architectural design against accepted quality criteria.
2. Identify tools and techniques to successfully manage a large-scale software project, including configuration management and version control.
3. Associate a range of appropriate tools with the development of a solution to a real-world problem.
4. Demonstrate standards, tools and techniques for assuring software quality.

Assignment Support and Academic Integrity

Spelling, Punctuation, and Grammar:

You are expected to use effective, accurate, and appropriate language within this assessment task.

Academic Integrity:

The work you submit must be your own, or in the case of groupwork, that of your group. All sources of information need to be acknowledged and attributed; therefore, you must provide references for all sources of information and acknowledge any tools used in the production of your work. We use detection software and make routine checks for evidence of academic misconduct.

It is your responsibility to keep a record of how your thinking has developed as you progress through to submission. Appropriate evidence could include version-controlled documents, developmental sketchbooks, or journals. This evidence can be called upon if we suspect academic misconduct.

If using Artificial Intelligence (AI) tools in the development of your assignment, you must reference which tools you have used and for what purposes you have used them. This information must be acknowledged in your final submission.

Unable to Submit on Time?

Student will be graded as ZERO if they are unable to submit on time.

Administration of Assessment

Module Leader Name: Thirunageswari Vija Kumaran

Module Leader Email: thirunageswari.maran@newinti.edu.my

Assignment Category: Portfolio

Attempt Type: Standard

Component Code: CW1

Overall Assessment Components :

Component	Type	Marks
a. System development	Group Work (Individually assessed)	40%
b. Individual Portfolio	Individual	50%
c. Viva	Individual	10%
Total Marks		**100%**

a. System Development Marks Allocation – 40%

Criteria	Descriptions	Marks
Functional Requirements	Implementation of Student, Lecturer and Admin modules	10
System Architecture Implementation	Proper MVC / 3-tier implementation	5
Database Design & Integration	Normalized database, relationships implemented	5
User Interface & Usability	Consistent UI/UX, navigation, responsiveness	5
System Features & Innovation	Additional features, automation, analytics	5
Code Quality & Structure	Modularity, naming conventions, maintainability	5
Team Contribution Evidence	Git commits, individual coding evidence	5
Total Marks		**40 Marks**

b. Individual Portfolio Marks Allocation – 50%

Criteria	Descriptions	Marks
Introduction & Problem Analysis	Project overview, Problem background, Objectives & scope	5 marks
Architecture Design & Evaluation	Architecture diagram, Design decisions. Evaluation using quality attributes: Performance, Security, Scalability, Maintainability	10 marks
Project Management & Configuration Management	SDLC model justification, Gantt chart, Version control usage, Configuration management strategy	10 marks
Tools & Technology Justification	C#, ASP.NET, Visual Studio, SQL Server, Framework justification	5 marks
System Design Documentation	Required diagrams: Use Case Diagram, ER Diagram, UML Diagram	5 marks
Development Documentation	Module explanation, Screenshots, Individual contribution	5 marks
Software quality assurance & Testing	Quality standards, Testing strategy, Test cases & results	7 marks
Evaluation & Reflection	System evaluation, Challenges, Future improvements	3 marks
Total Marks		**50 marks**

c. VIVA Assessment Rubric – 10%

Criteria	Marks
Understanding of System Architecture	3
Explanation of Individual Contribution	3
Software Engineering Knowledge	2
Ability to Answer Technical Questions	2
Total Marks	**10 Marks**

a. Marking Criteria : System Development

Criteria						Total Marks
	0-2	3-4	5-6	7 - 8	9 - 10	
Functional Requirements	No working modules. Login or core system not functioning. Student, Lecturer, or Admin modules missing. System cannot perform basic operations.	Only partial module implementation. Some pages exist but major functions fail. Limited CRUD operations. Workflow incomplete.	Core modules available (Student, Lecturer, Admin). Basic operations such as registration or record viewing work but contain errors or missing validations.	Most functional requirements implemented correctly. Modules interact properly. Application workflows operate with minor issues only.	All required modules fully implemented and integrated. Complete workflows (login, role access, data management, reporting). Stable and reliable system behaviour.	**10**
System Architecture	**0-1**	**2**	**3**	**4**	**5**	**5**
	No clear architecture. Code written without separation of concerns.	Partial MVC/3-tier structure attempted but inconsistent implementation.	Basic MVC separation implemented (Models, Views, Controllers identifiable).	Proper MVC architecture followed with organised layers and routing.	Full MVC/3-tier architecture correctly implemented with maintainable structure and scalability consideration.	
Database Design & Integration	**0-1**	**2**	**3**	**4**	**5**	**5**
	Poor database structure. Missing relationships or incorrect storage.	Basic tables created but limited normalization.	Correct tables and relationships implemented. CRUD operations functional.	Normalized database with proper foreign keys and constraints.	Well-designed normalized database supporting efficient queries and system scalability.	
User Interface & Usability	**0-1**	**2**	**3**	**4**	**5**	**5**
	Interface missing or difficult to use. Navigation unclear.	Basic interface with poor layout consistency.	Functional UI with acceptable navigation.	Consistent layout, responsive design, user-friendly navigation.	Professional UI/UX design with clear workflow, responsiveness, and usability principles applied.	
System Features & Innovation	**0-1**	**2**	**3**	**4**	**5**	**5**
	Only minimum requirements implemented.	Minor additional features attempted.	Some enhancement features included.	Useful advanced features added (reports, automation, notifications).	Innovative system enhancements such as analytics dashboards, chatbot, automation, or smart features.	
Code Quality & Structure	**0-1**	**2**	**3**	**4**	**5**	**5**
	Disorganized code, poor naming, duplicated logic.	Basic coding structure but inconsistent practices.	Acceptable structure with readable code.	Modular design with good naming conventions and reuse.	Clean, maintainable, well-documented professional-quality code.	

Team Contribution Evidence	**0-1**	**2**	**3**	**4**	**5**	**5**
	No evidence of contribution.	Limited commits or unclear ownership	Regular commits showing contribution.	Clear individual module responsibility demonstrated.	Strong Git history, collaboration evidence, and clear ownership of developed features.	

b. Marking Criteria : Individual Portfolio

Criteria						**Total Marks**
	0 - 1	**2**	**3**	**4**	**5**	
Introduction & Problem Analysis	Very minimal or missing content.	Weak problem analysis and unclear objectives.	Basic explanation but lacks depth or clarity.	Good explanation with minor missing details.	Clear project overview, strong problem justification, well-defined objectives and scope.	**5**
Architecture Design & Evaluation	**0-2**	**3-4**	**5-6**	**7-8**	**9-10**	**10**
	Missing or incorrect architecture design.	Incomplete design and weak explanation.	Architecture provided but evaluation limited.	Clear design with reasonable evaluation.	Complete architecture diagram with strong evaluation using quality attributes.	
Project Management & Configuration Management	**0-2**	**3-4**	**5-6**	**7-8**	**9-10**	**10**
	Missing planning or configuration management.	Weak evidence of project management practices.	Basic planning and limited management explanation.	Good planning with minor gaps.	Strong SDLC justification, detailed Gantt chart, clear configuration & version control strategy.	
Tools & Technology Justification	**0-1**	**2**	**3**	**4**	**5**	**5**
	No justification provided.	Weak justification.	Basic explanation only.	Adequate justification.	Strong justification aligned with system requirements.	
System Design Documentation	**0-1**	**2**	**3**	**4**	**5**	**5**
	Missing diagrams.	Incomplete diagrams.	Diagrams present but lack detail.	Mostly correct diagrams with minor issues.	All diagrams complete, correct, and professional.	
Development Documentation	**0-1**	**2**	**3**	**4**	**5**	**5**
	Missing development evidence.	Minimal documentation.	Basic explanation provided.	Good documentation with minor gaps.	Clear module explanation, screenshots, and individual contribution evidence.	
Software Quality Assurance & Testing	**0**	**1-2**	**3-4**	**5**	**6-7**	**7**
	No testing presented.	Minimal testing evidence.	Basic testing shown.	Adequate testing coverage.	Comprehensive testing strategy, quality standards, and evaluated results.	
Evaluation & Reflection	**0**	**1**	**2**	**2.5**	**3**	**3**
	Reflection missing or irrelevant.	Limited reflection; mostly descriptive with minimal evaluation.	Adequate reflection provided but analysis lacks depth or critical insight.	Clear evaluation and reflection with good discussion of challenges and possible improvements.	Critical system evaluation with deep reflection on learning, challenges, and well-justified future improvements.	

c. Marking Criteria : VIVA Presentation

Criteria						Total Marks
	0	1	2	2.5	3	
Understanding of System Architecture	Unable to explain system architecture.	Limited understanding; explanation partially correct.	General understanding shown but explanation lacks depth.	Very good understanding with minor clarification needed.	Demonstrates comprehensive understanding of system architecture, components, workflow, and design decisions confidently.	**3**
Explanation of Individual Contribution	**0**	**1**	**2**	**2.5**	**3**	**3**
	Unable to identify own contribution.	Minimal explanation of individual contribution.	Contribution explained but technical details limited.	Good explanation with minor missing details.	Clearly explains individual role, modules developed, responsibilities, and technical ownership.	
Software Engineering Knowledge	**0**	**0.5**	**1**	**1.5**	**2**	**2**
	No relevant knowledge demonstrated.	Limited knowledge shown.	Basic understanding demonstrated.	Good theoretical knowledge with minor gaps.	Strong understanding of SDLC, architecture, testing, and engineering principles.	
Ability to Answer Technical Questions	**0**	**0.5**	**1**	**1.5**	**2**	**2**
	Unable to answer technical questions.	Weak answers with limited understanding.	Partially correct responses.	Mostly correct answers with minor hesitation	Answers confidently with accurate technical justification.	